

Object Centric Networking, from Databases to Networked Objects

J. Augé, M-O Buob, and S. Fdida

LINCS – September 14, 2015



Agenda

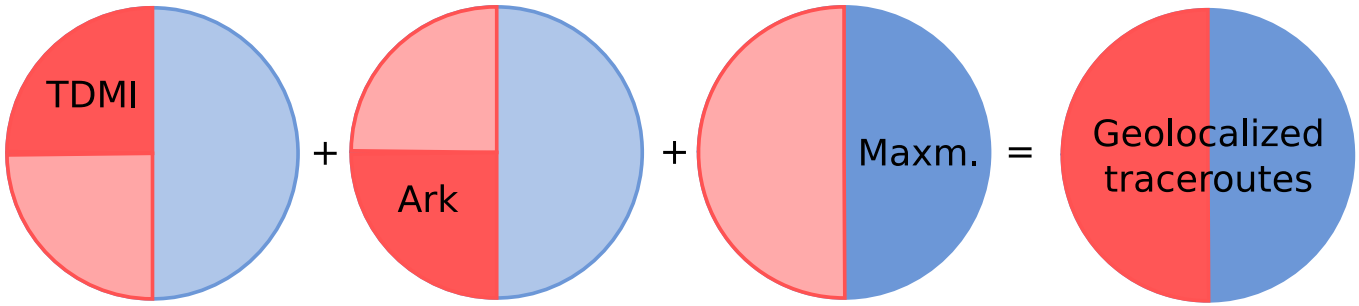
1. What is Manifold?
2. Some real use cases.
3. How to use Manifold?
4. How does work Manifold?

Part. 1

Manifold in a nutshell.

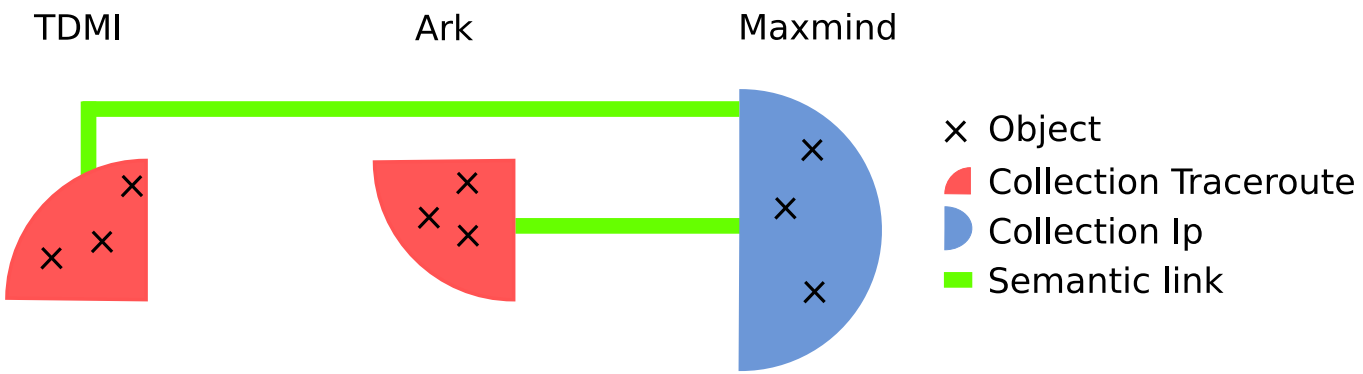
1. Overview.
2. Proposal.
3. Outcomes.

Overview



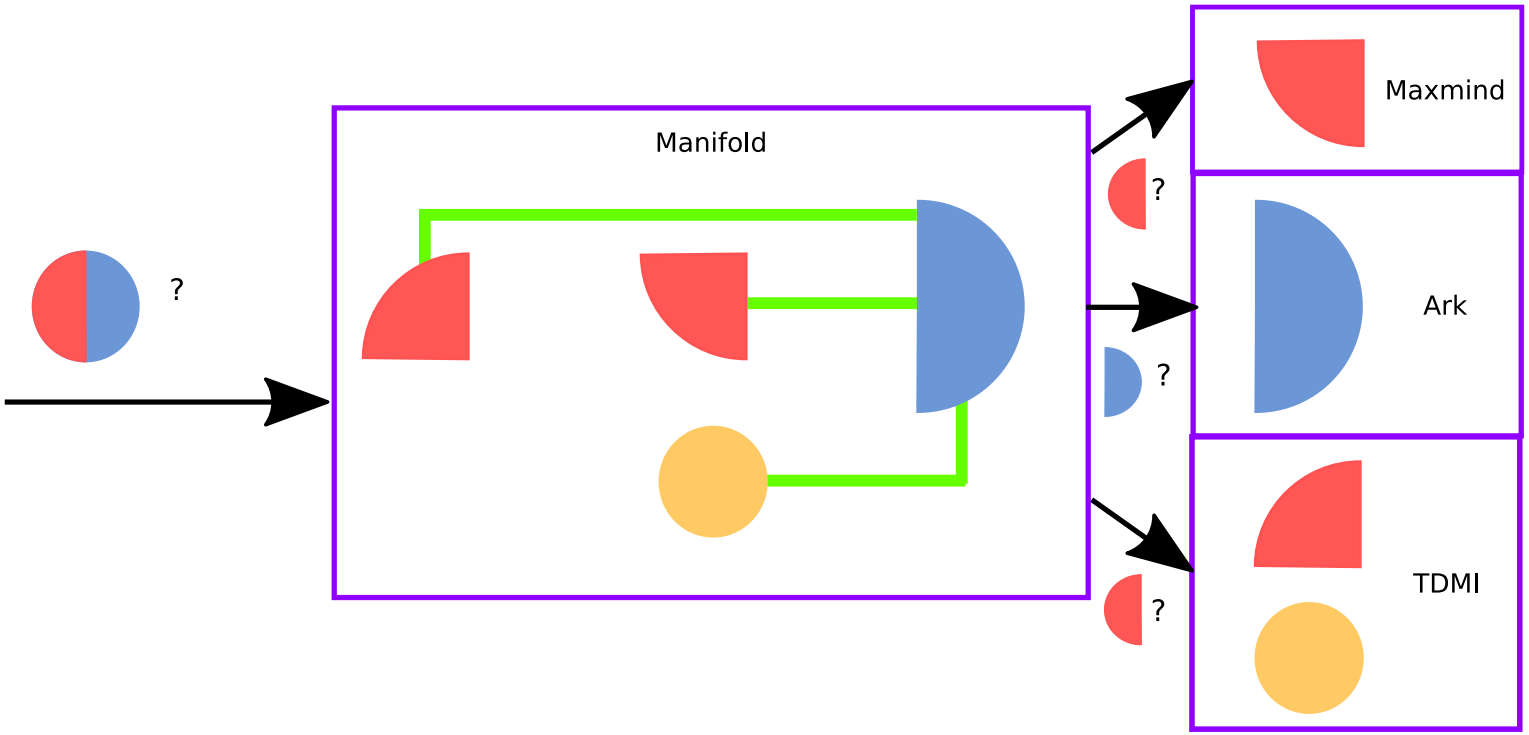
- *Goal:* query heterogeneous objects offered by the network.
- *Challenges:* managing heterogeneity (location + structure + storage).
- *Example:* TDMI and Ark databases provide traceroute measurements. Maxmind API allows to geolocalize IP addresses.

Proposal



- *Objects* (= instance) belongs to a collection.
- *Collection* (= type) describe the structure of the data.
- *Semantic links* may connect collections.
- Collections are responsive to *Manifold query language*.

Outcomes



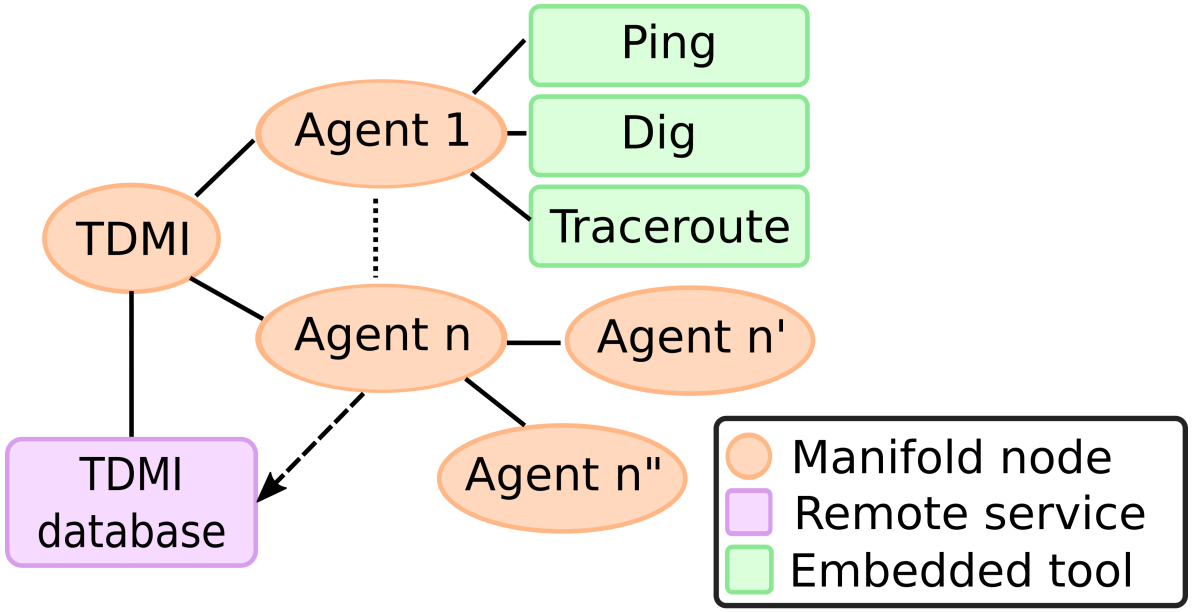
- Manifold allows to query/gather/combine *transparently* networked objects.
- Manifold = network paradigm + programming language.

Some real use cases.

1. Federating testbeds (ex: Onelab portal).
2. Distributed measurement platform (ex: TDMI).
3. Monitoring (ex: tophat).
4. Scientific experiments (ex: anycast).

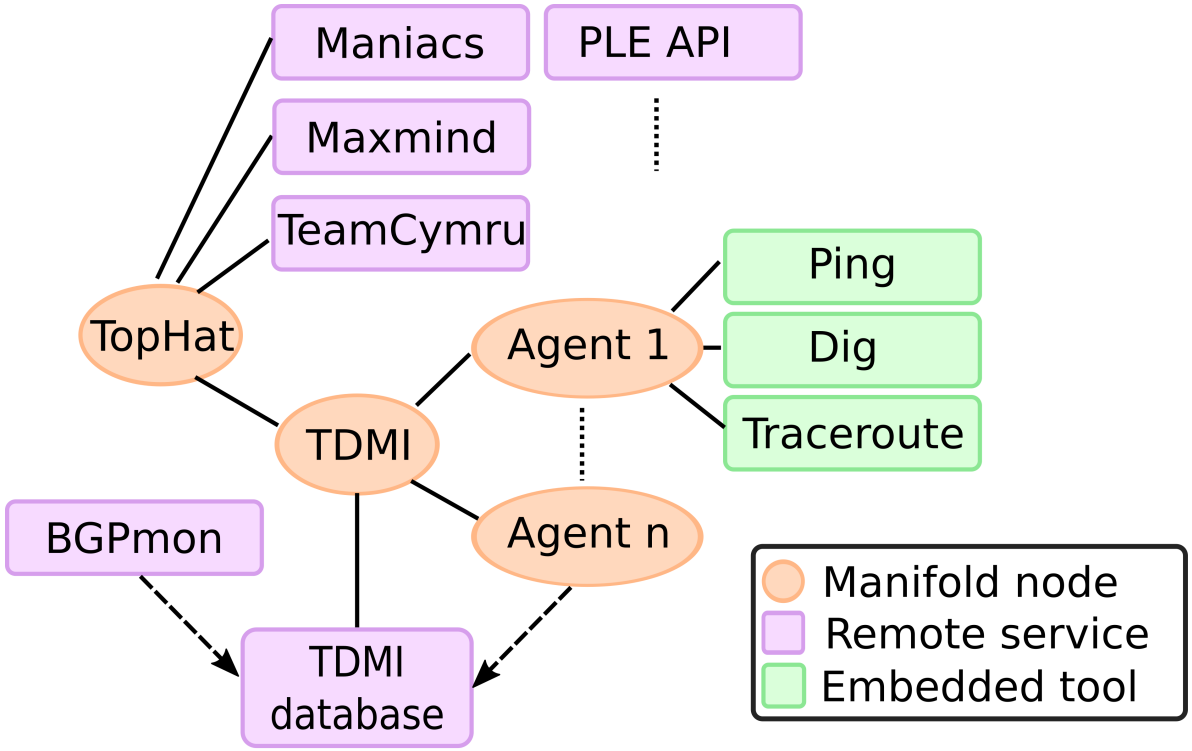
Note: these examples could be extended with additional services and tools.

TDMI



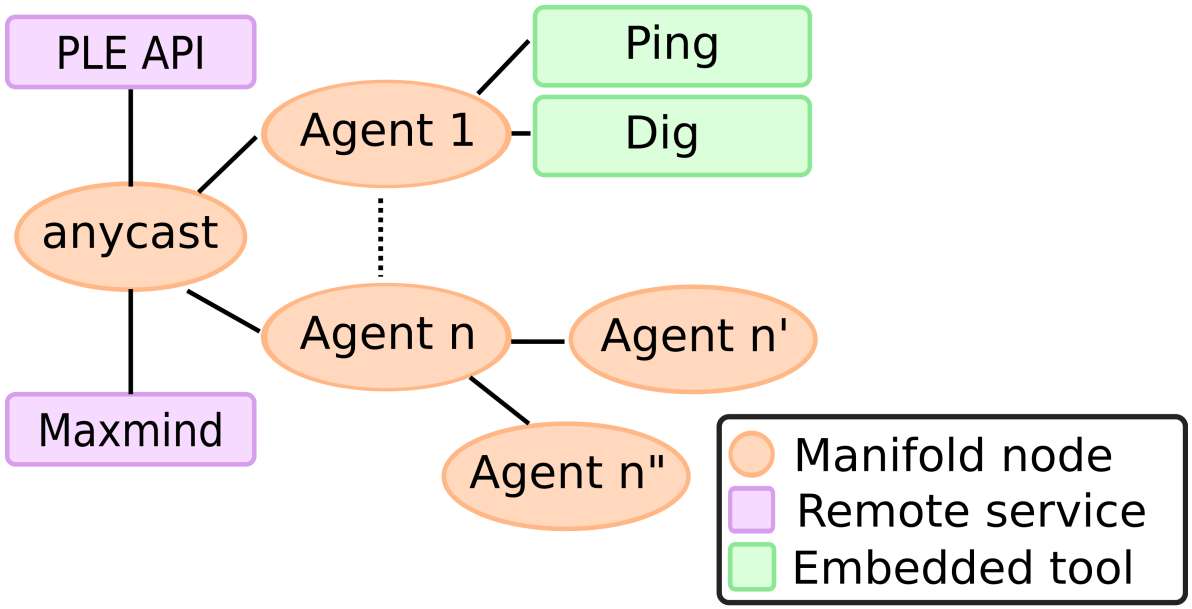
- TDMI overlay is a dynamic tree and is self-organized. It is running over PLE.
- They push every 5 minutes traceroute measurements in the TDMI database.
- Agents can perform on-demand measurements.

TopHat



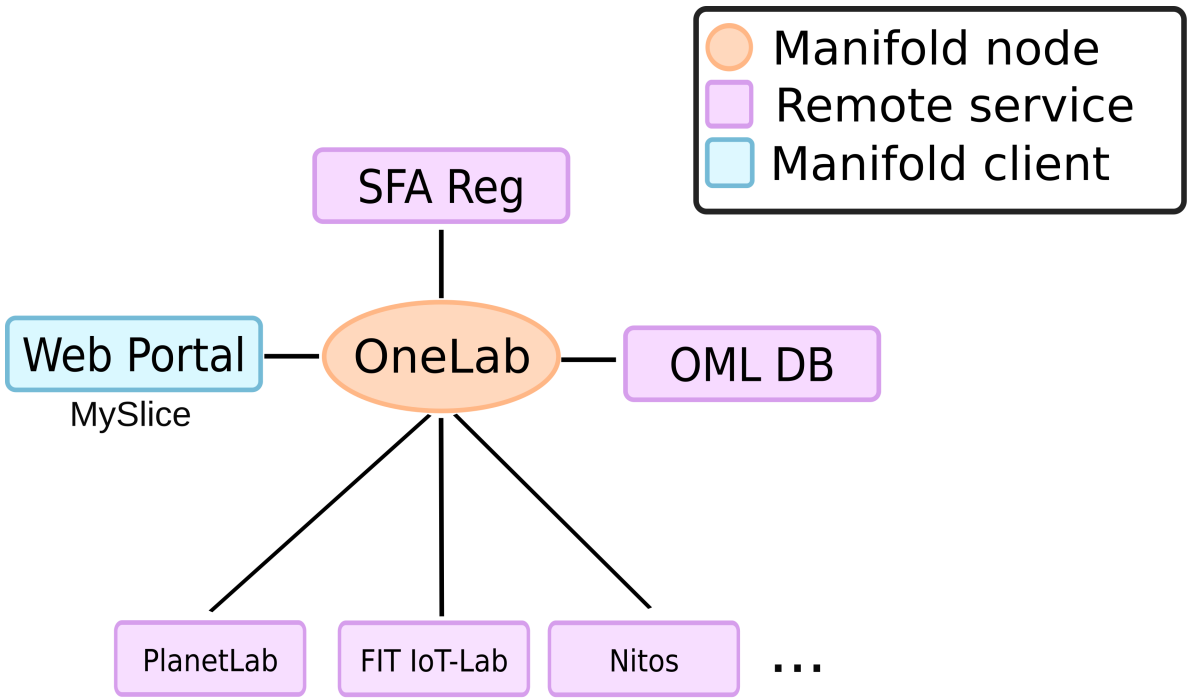
- TopHat enriches TDMI measurement thanks to tiers services.
- *Maniacs:* AS classification; *MaxMind:* IP geolocation; *TeamCymru:* IP-to-AS mapping; *BGPMon:* BGP dumps; *PLE:* Agent geolocation.

Anycast



- They pings a given target IP. The TTL bounds the distance between the agent and the anycast replica (discs).
- Agents are geolocalized (disc centers).
- $max(\#disjoint_discs) \leq \#replicas$

Onelab portal



Resource browsing and reservation, user account management, leases.

How to use manifold?

1. Data model.
2. Querying objects (data consumer side)
3. Publishing object (data producer side)

Goal

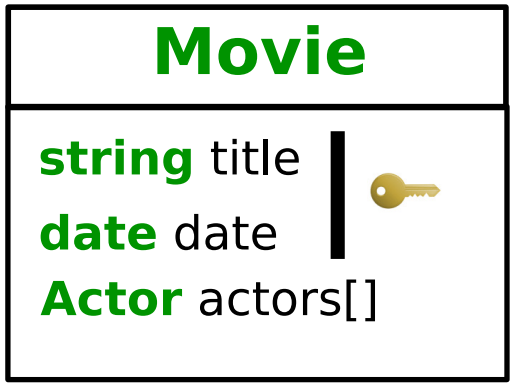
Manifold aims at offering a query language over networked objects.

1. How to describe such objects?
2. How to establish semantic links?
3. Which language to query an object?
4. How to make the network transparent to the user?

Data model: objects

Design inspired from Oriented Object Programming and databases.

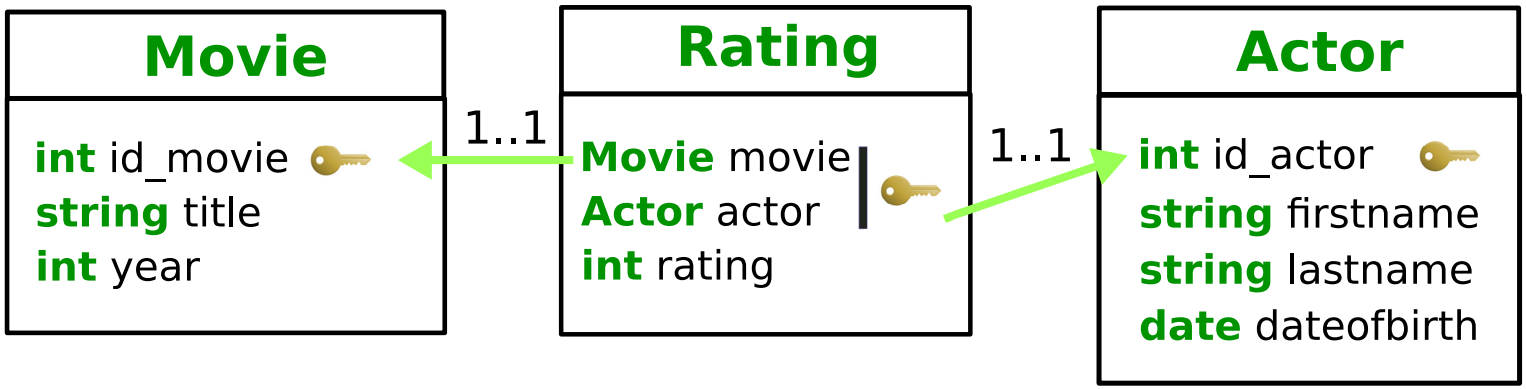
- Each collection is identified by a *name*.
- A collection is made of *typed fields*.
- A *key* (= subset of fields) identifies an object.
- A field may be an array (ex: a list of actors).



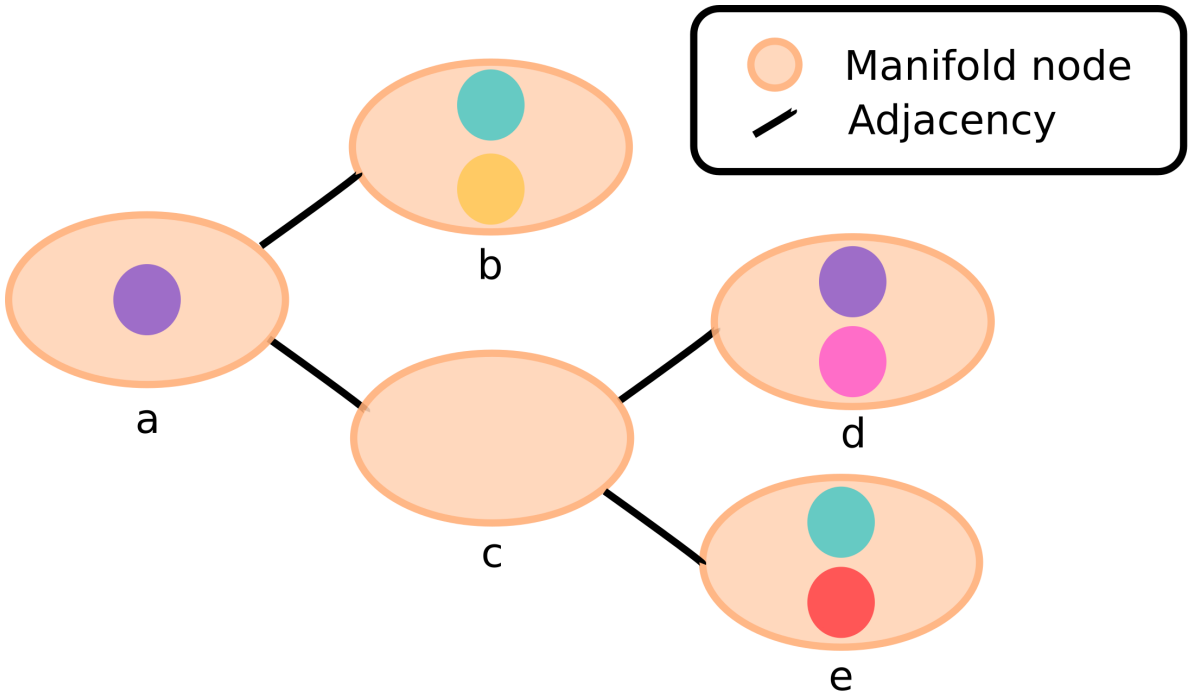
Rk: the naming *should* be consistent in the whole ecosystem.

Data model: semantic links

- *Semantic links* are deduced according to *typing*.
- An object refers to another one using its key.
- Links are *directed* and carry a *cardinality*.

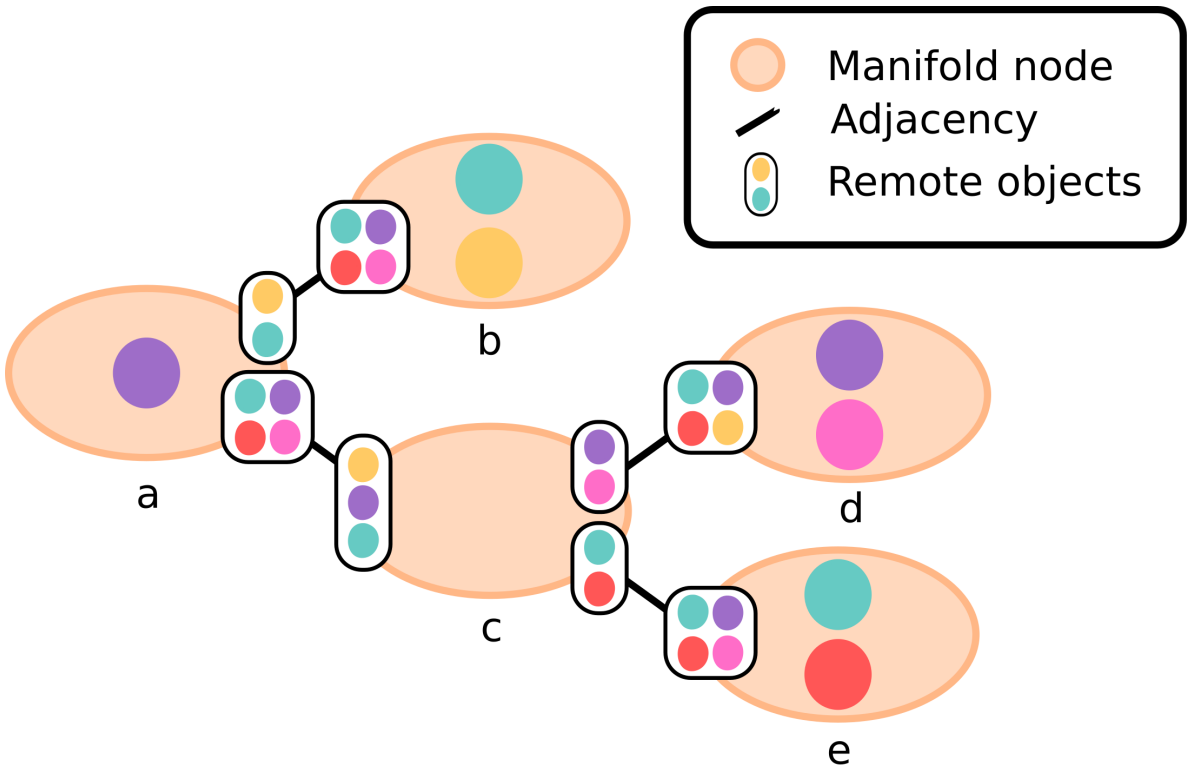


Big picture (1/7): collections



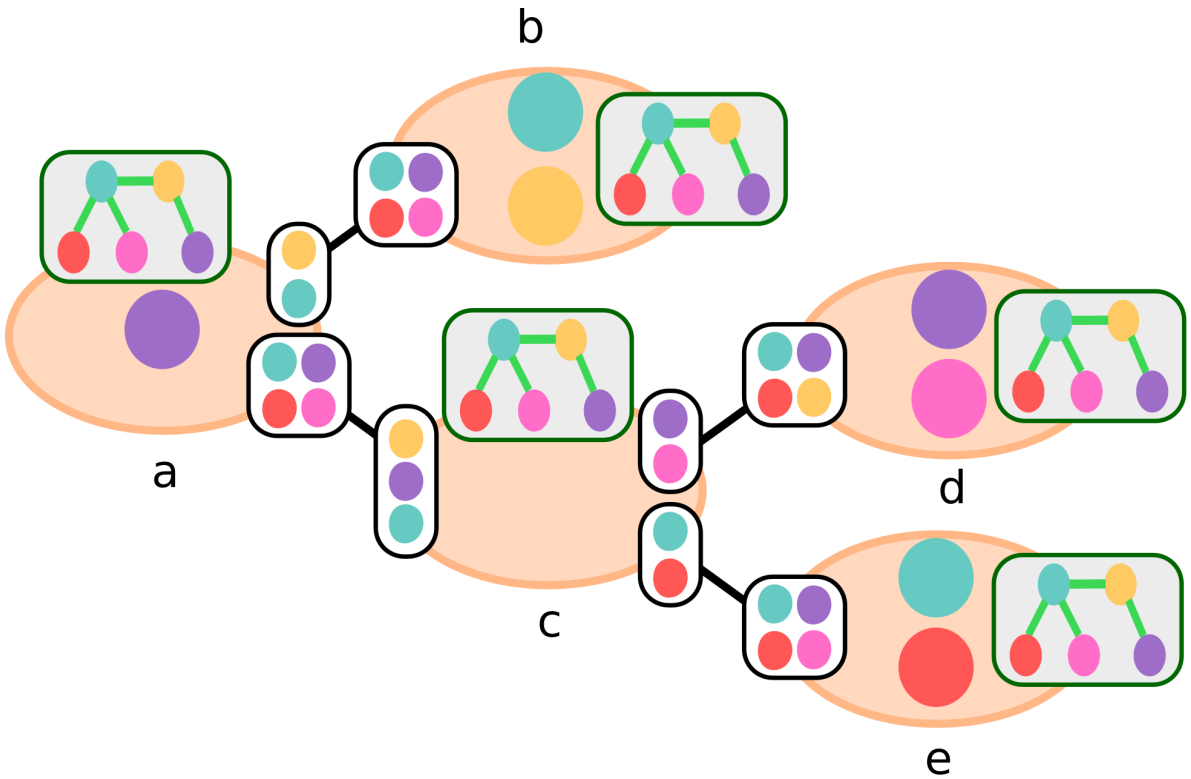
Manifold nodes may own zero or more collections.

Big picture (2/7): announces



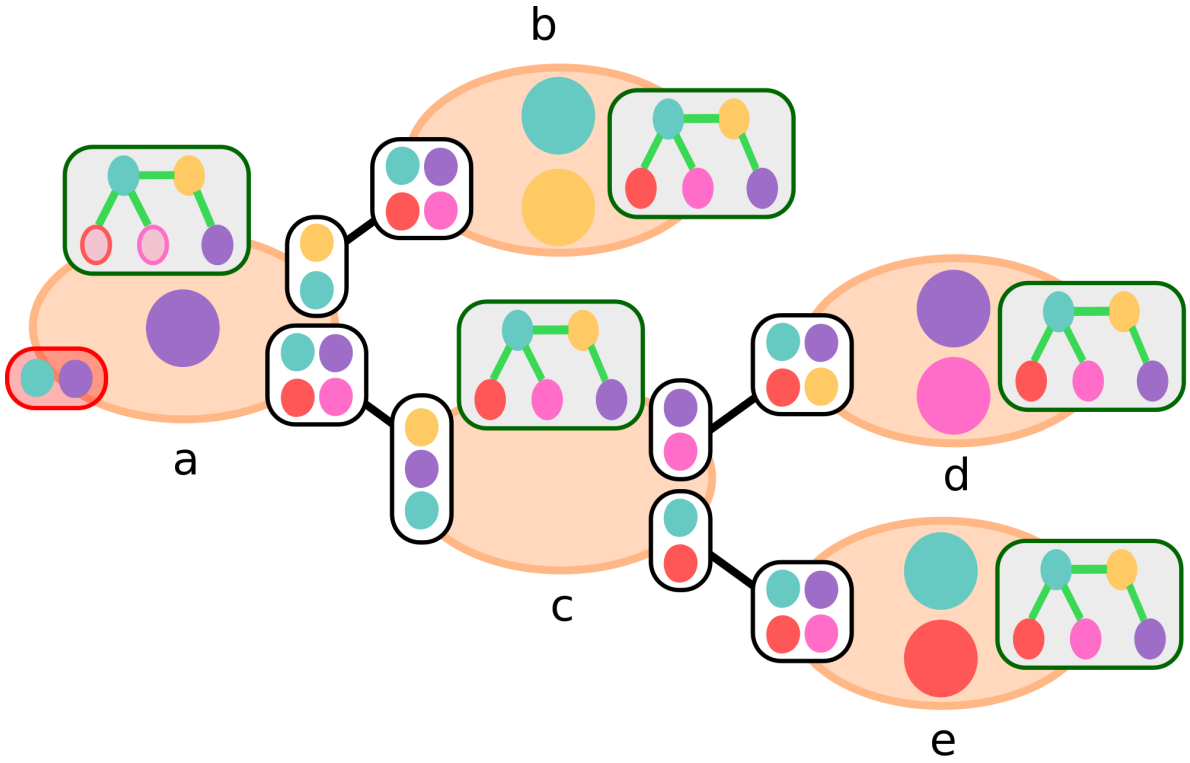
- They announces to their peers a subset of the objects they can reach.
- Like in BGP, announces can be ruled by a routing policy.

Big picture (3/7): routing table



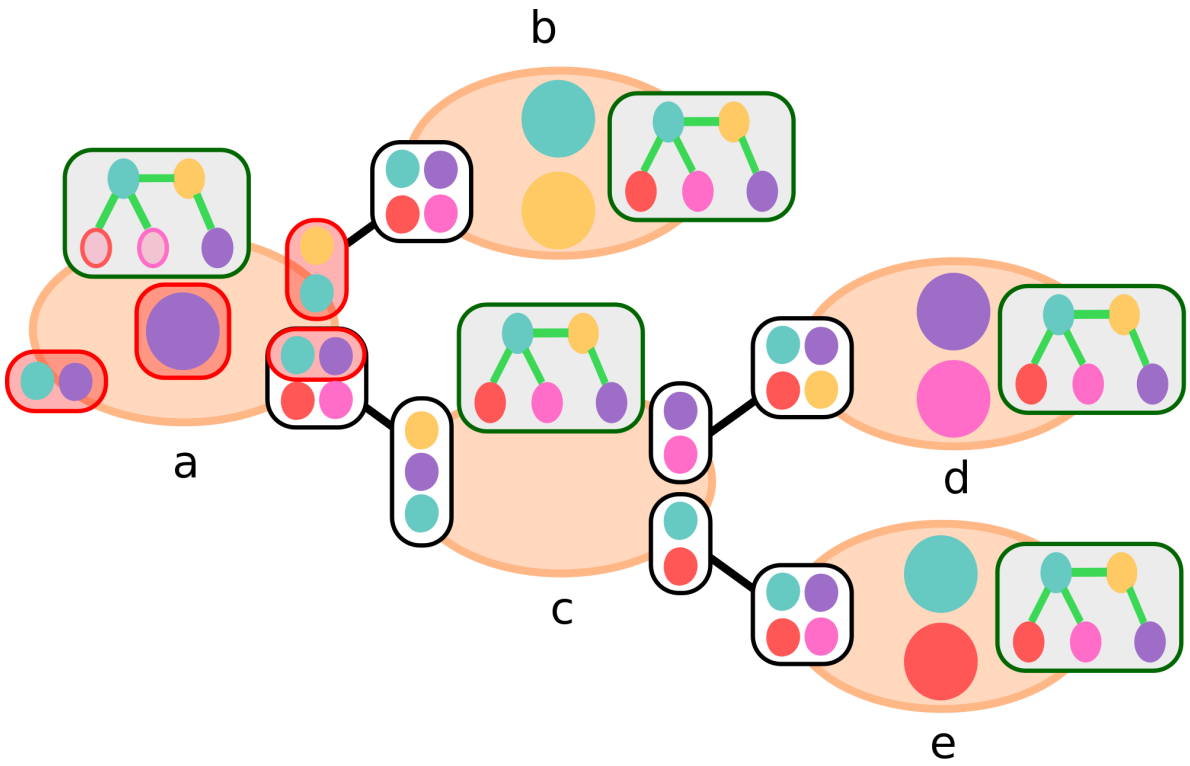
They rebuild internally their perception of the underlying *ontology* ($\approx$ routing table).

Big picture (4/7): query



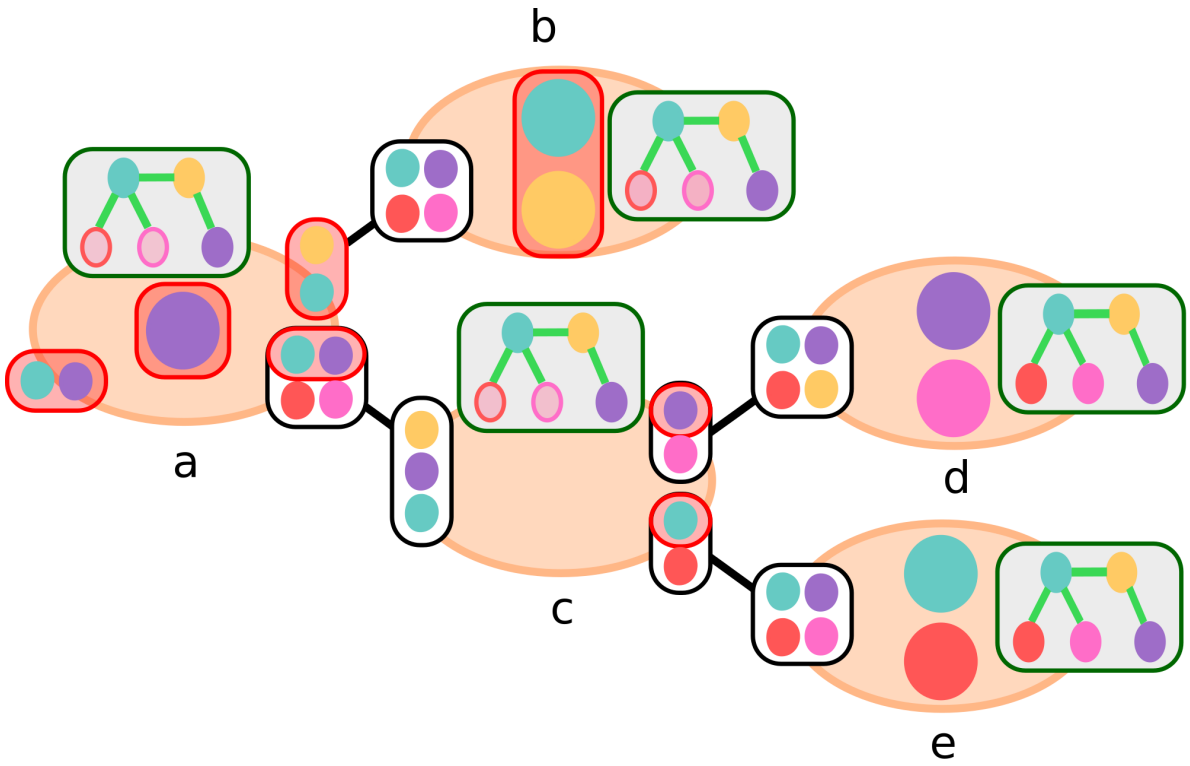
A q handled query triggers the computation of a query plan and leads to a *forwarding*. Relevant purple objects depends on the yellow retrieved items.

Big picture (5/7): query



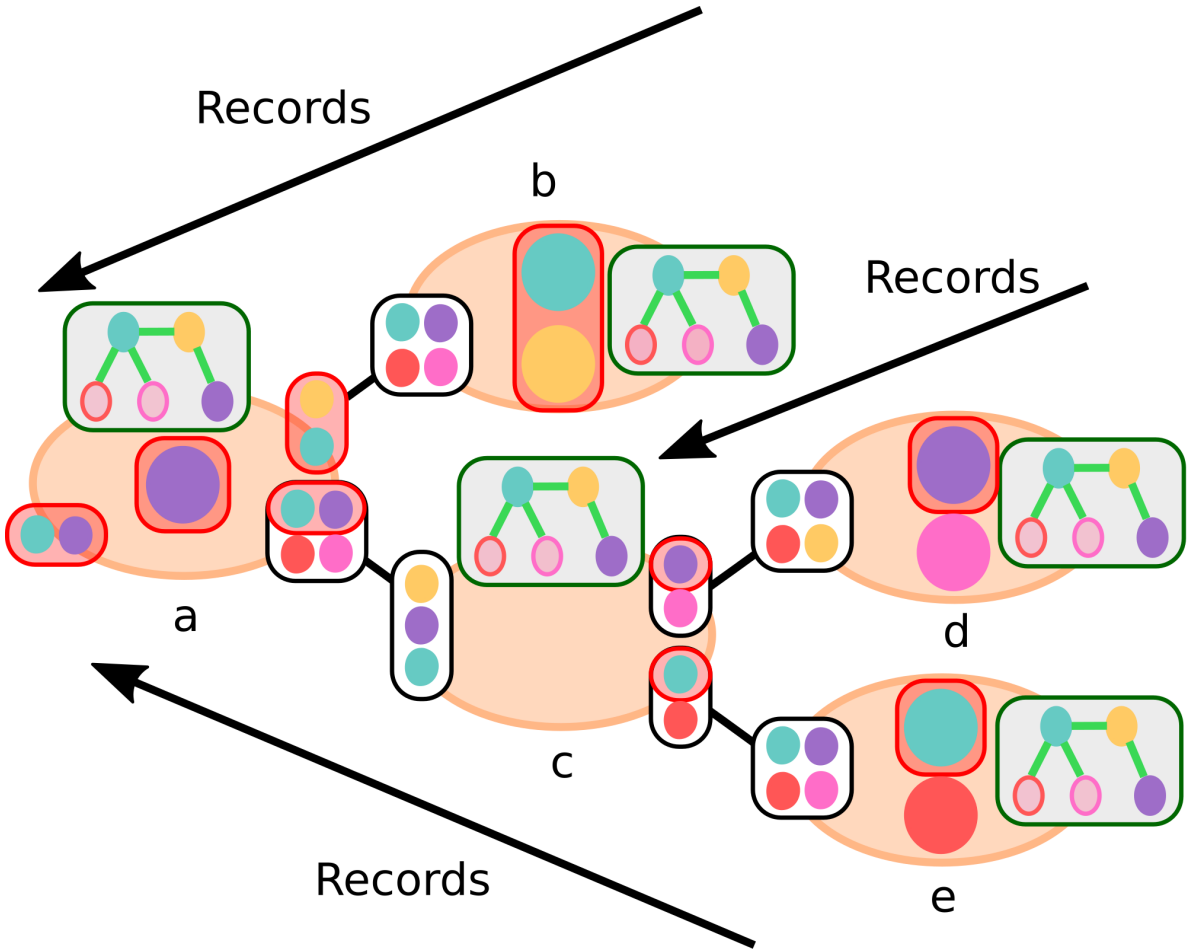
Peer providing objects of interest are queried.

Big picture (6/7): query



The query is forwarded (and updated) hop-by-hop.

Big picture (7/7): query



Providers return object of interest to the consumer(s). Records may be processed (e.g. combined, filtered) along the retrieval tree.

Queries in Manifold

Reminder: main relational algebra operators:

Symbol	Operation	SQL/Manifold operator
σ	Projection	`SELECT`
π	Selection	`WHERE`
`(T)`	From T	`FROM object_name`
$\bowtie$	Join	`LEFT OUTER JOIN`
$\cup$	Union	`UNION`

Transparent ecosystem $\Longrightarrow$

- User express its interests: σ, π, `(T)`
- *Logical independence:* $\bowtie$ and $\cup$ are deduced.
- *Physical independence:* data provider may be omitted.

Syntax (1/3)

- Queries are typed in `manifold-shell`.
  - `$ manifold-shell -z auth_method -u manifold_user`
  - `$ manifold-shell -z router -u foo@bar.com`
- It connects to a `manifold-router`, optionally configured according to a storage.
- CRUD (Create, Read, Update, Delete).
- Syntax based on SQL.

Syntax (2/3)

Create:

```
INSERT INTO [namespace:]object_name
SET field_1 = value_1 [, ..., field_n = value_n]
[AT (date | NOW)]
```

Read:

```
SELECT (field_1 [, ..., field_n] | *)
FROM [namespace:]object_name
[WHERE clause]
[AT (date | NOW)]
```

Syntax (3/3)

Update:

```
UPDATE [namespace:]object_name
SET field_1 = value_1 [, ..., field_n = value_n]
[WHERE clause]
```

Delete:

```
DELETE FROM [namespace:]object_name
[WHERE clause]
[AT (date | NOW)]
```

Some useful objects

Some virtual objects allow to retrieve the state of the router.

- They all belong to the `local:` namespace.
- `local:object`: objects learned by the router.
- `local:about`: version of manifold.
- `local:user`: manifold users.
- `local:platform`: manifold peers.

Example:

```
SELECT object_name, columns.name FROM local:object
```

Publishing objects in manifold (1/2)

Design inspired from BGP: you can reach what you learn.

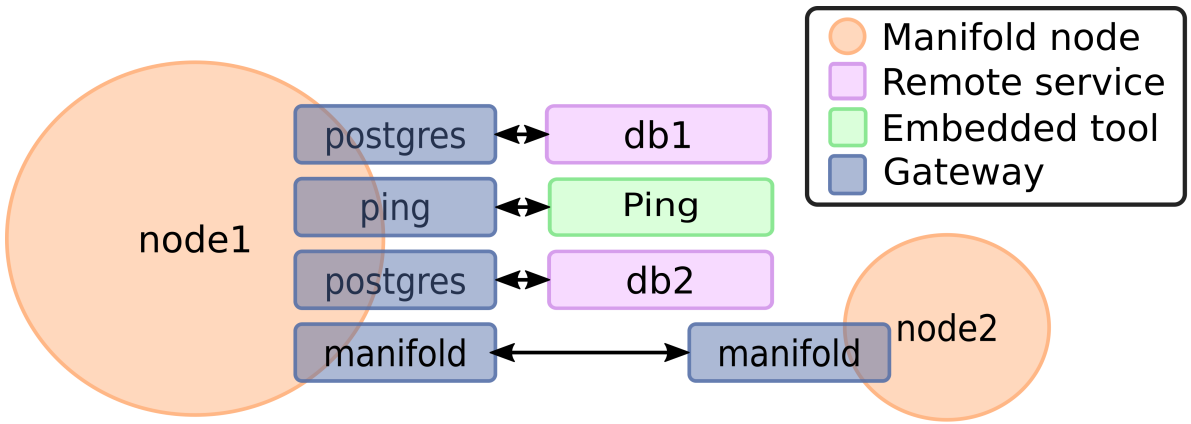
- A provider *announces* the collections it provides.
- It may attach *capabilities* ($\approx$ policies) to specify which relational operations are supported.
- *Local* fields can only refer to its own objects.
- *Const* fields are read-only.

Example:

```
class Movie {
    local const int id_movie;
    const string title; /**< Title of the movie. */
    const int year;     /**< Year of the movie.  */
    CAPABILITIES(Selection, Projection);
    KEY(id_movie);
};
```

Publishing objects in manifold (2/2)

- *Problem:* how to speak manifold to a storage which is not manifold compliant?
  - *Solution:* write adaptors, called *gateways*, translating queries and records.



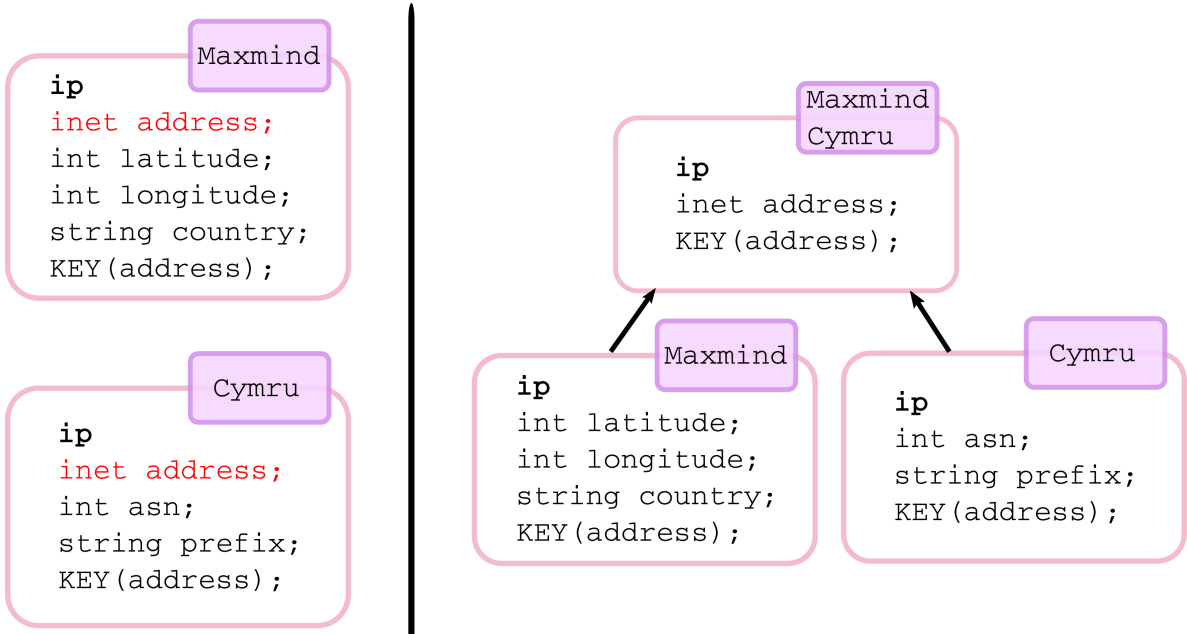
- Many gateways already exist: (`manifold/gateways`)
  - postgres, csv, maxmind, teamcymru, traceroute, ping, dig, ...
- Template available in `manifold/gateways/template.py`

How does work manifold?

1. Routing (database normalization).
2. Forwarding computing an optimized query plane.
3. Manifold packets.
4. Manifold flows.

Managing heterogeneity with normalization

- Normalization = well-known algorithm[1] from databases pruning redundancy.
- We assume that the underlying ontology conforms to a 3NF schema.
- 3NF schema = routing table = approximation of the ontology.



Before normalization (announces)

After normalization (routing table)

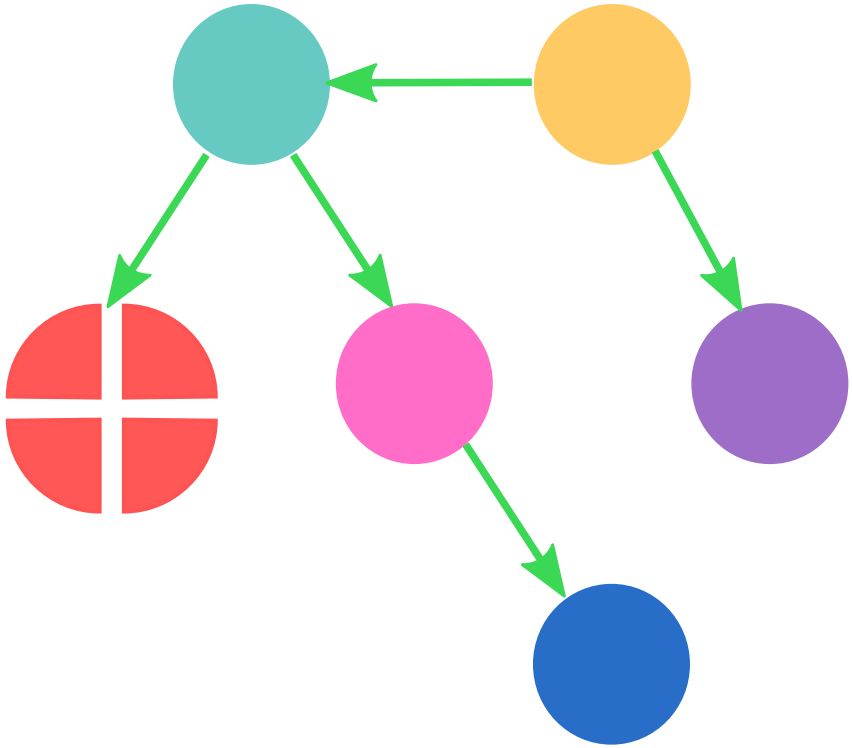
[1] http://elm.eeng.dcu.ie/~ee221/EE221-DB-7.pdf

Query plane: overview

An incoming query $q(\sigma, \pi, T, \ldots)$ triggers *query plan* computation.

1. Get object name and dimensions of interest: σ, π, `(T)`.
2. Query plan = (minimal) combination of collections.
   1. Find covering tree in the 3nf graph, rooted in T, and reaching all the fields involved in σ and π.
   2. Traversed `1..1` arcs $\Longrightarrow$ $\bowtie$.
   3. Traversed `1..N` arcs $\Longrightarrow$ `SUBQUERY`.
   4. Multiple peers provides a required object $\Longrightarrow$ $\cup$
   5. Install π and σ.
3. Optimize query plan.

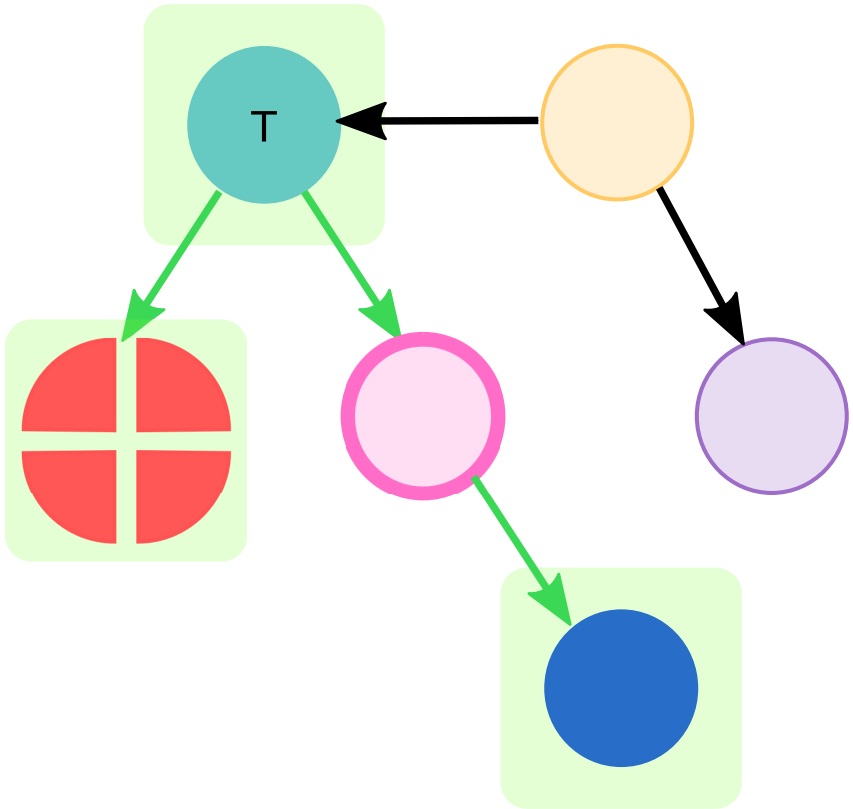
Building the query plane (1/4)



3nf schema. Red object is provided by several peers.

Building the query plane (2/4)

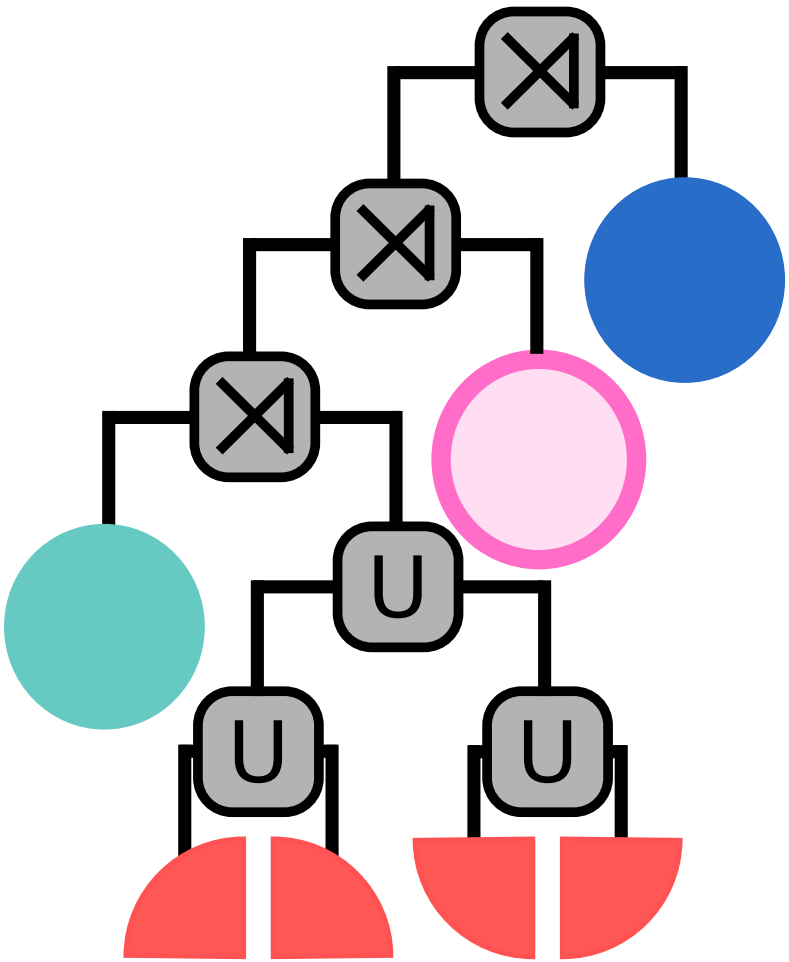
$q = (\sigma = \,$  $, \pi = \,$  $, T = \,$  $)$



- Find dimensions of interest (σ, π).
- Pink item is required to combine cyan and blue objects.

Building the query plane (3/4)

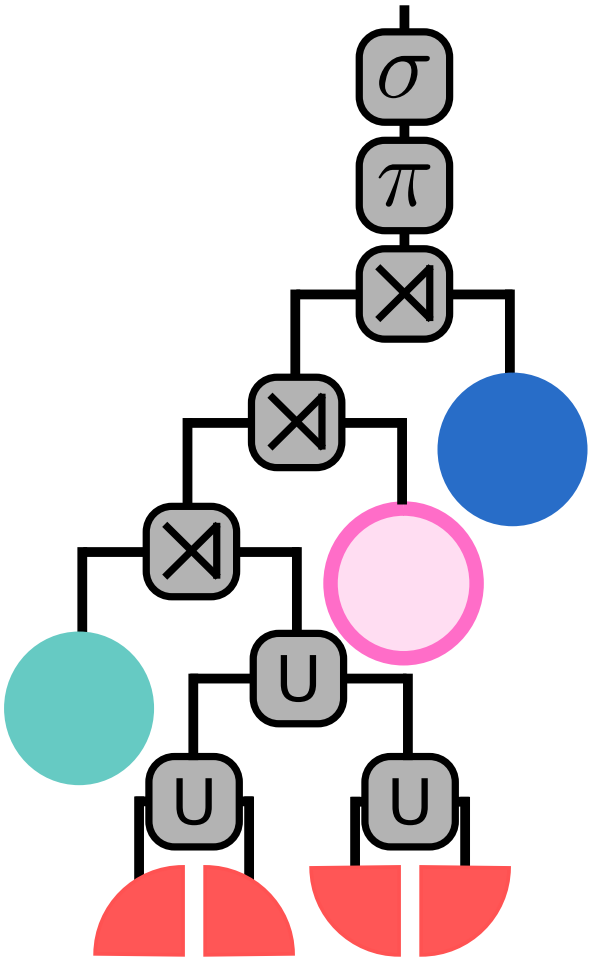
$q = (\sigma = \text{◗}, \pi = \text{◓}, T = \text{●})$



Deduce the tree of $\cup$ and $\bowtie$.

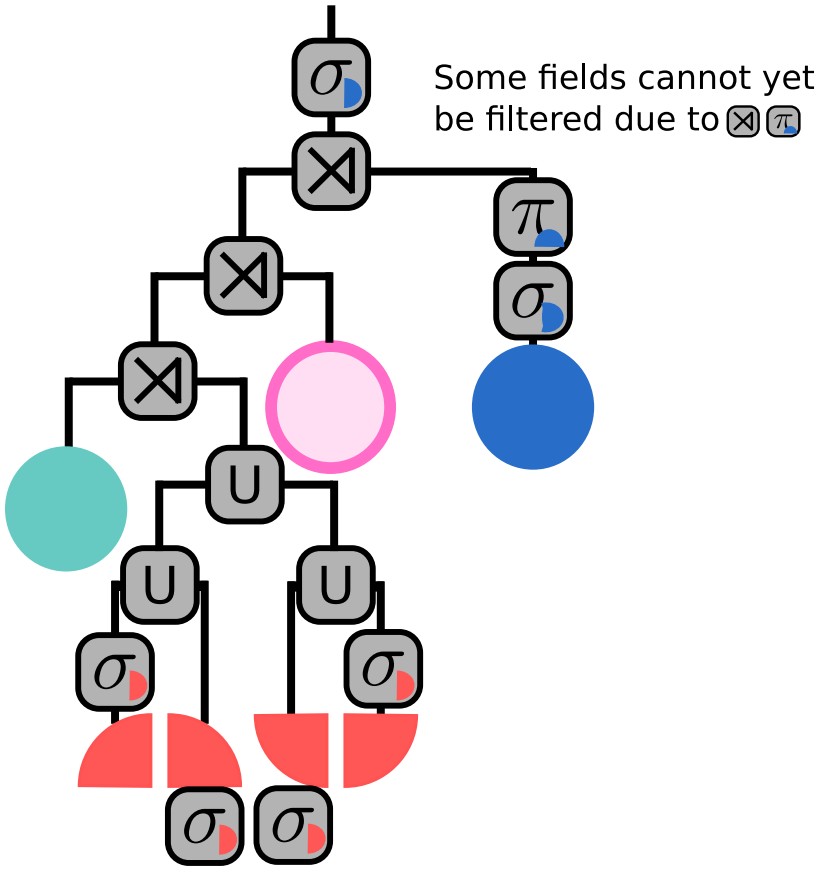
Building the query plane (4/4)

$q = (\sigma = \text{}, \pi = \text{}, T = \text{  })$



Add filters (π, σ) of q on the top of the query plane.

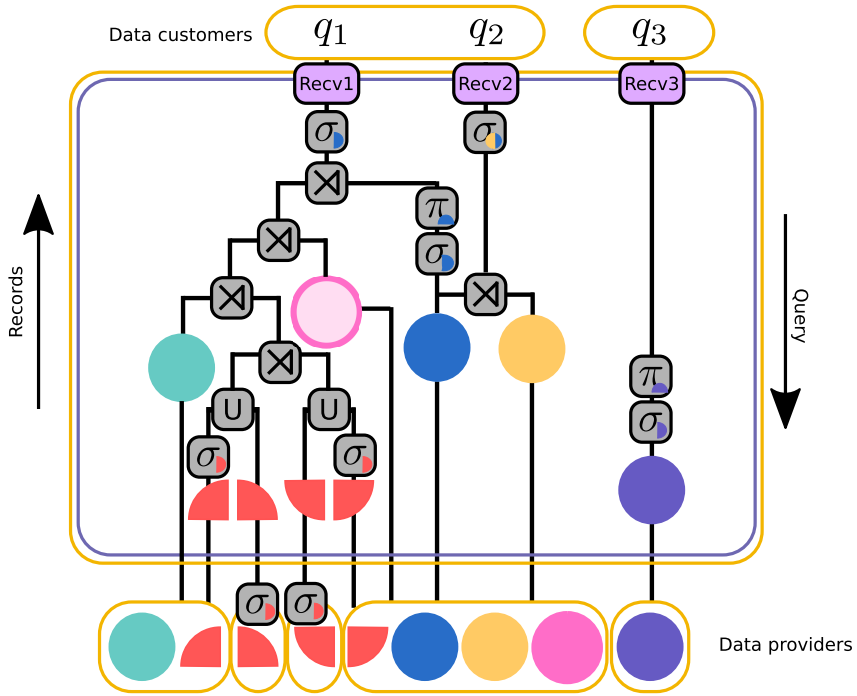
Optimizing the query plane



- Optimize the query plan according to well-known transformations[a].
- *Idea:* filtering ASAP to avoid useless processings.
- If a peer supports an operation, delegate to it the corresponding operator.
- Program a distributed query plane over the network.

[a] https://en.wikipedia.org/wiki/Relational_algebra

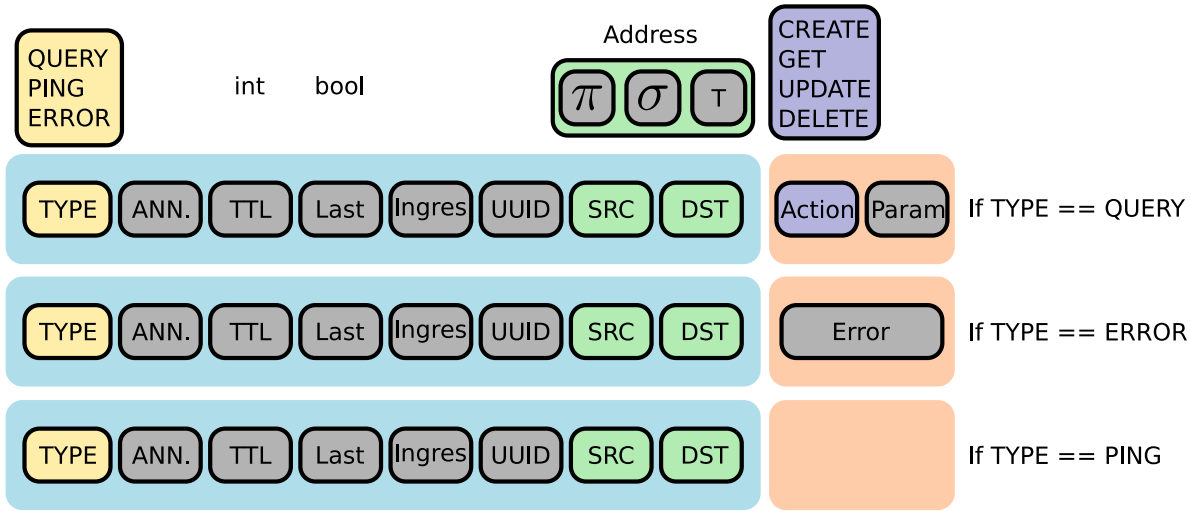
Running several query plan



- Pending queries are gathered in a single *operator graph*.
- Receiver $\approx$ socket.
- Streaming queries (e.g. q_1 and q_2) may share streams.
- Node having collected all their records can be destroyed.

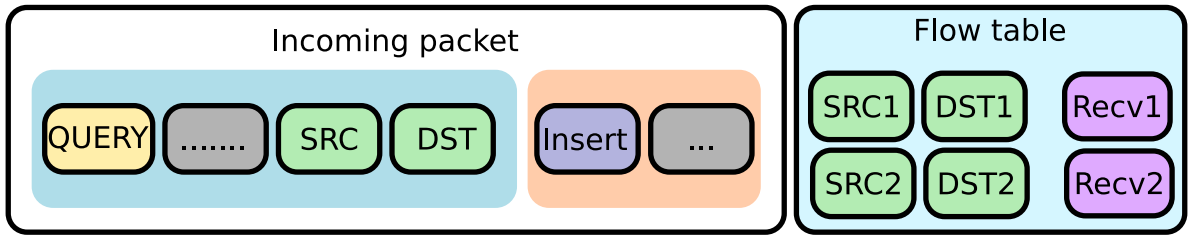
Toward a ICN paradigm: packets

Packets are handled by routers, operators, and gateways.



- *Addresses* are pieces of query.
- *Ingres* memorizes where a packet entered to avoid loops.
- *Annotations* carry for example credentials.
- *Records* are carried by (QUERY, INSERT) packets.

Toward a ICN paradigm: flows



- (QUERY, INSERT) packets can either:
  - A new INSERT query ($\Longrightarrow$ new query plan).
  - Records resulting from a query ($\Longrightarrow$ pipe them in the operator graph).
- *Problem:* determining INSERT packet forwarding (up/down?).
- *Solution:* tests whether the packet matches a pending flow.

Conclusion

Manifold = *new ICN paradigm* inspired from *databases*.

- It *federates* heterogeneous objects offered by the network.
- It's *generic* (as illustrated in our *real* use cases).
- It's still an ongoing work[2], contributors are welcome.

Thank you! Questions?

[2] `git clone ssh://tophat@git.top-hat.info/tophat.git`